Personal Finance System: Zero-Friction Expense Capture, Budget Control and AI Audit

PF-2609-PY-003 v1	*Python / Supabase (PostgreSQL) / GitHub Actions / iOS Shortcuts / Claude · First documented September 2026 · v1, October 2026* *Status: in operation (personal production) and released as an open-source template. Track 1 (Capture), Track 2 (Ingestion), Track 3 (Audit).*

2. Tool Stack

Python 3.12	**PostgreSQL 17 (Supabase)**	**GitHub Actions**	**iOS Shortcuts**	**Supabase REST + Storage**	**Supabase Vault + pg_net**	**Telegram Bot API**
Qwen3-VL (DashScope)	**Claude + MCP connector**	**requests 2.31+**	**pytest 8+**	**PyYAML 6+**	**zoneinfo (stdlib)**	**SQL views + triggers**

3. Project Objective

Replace a manually maintained spreadsheet and a habit that people abandon after a few weeks with a closed-loop personal finance system with **zero-friction capture**: one tap on a phone records an expense, the system validates it, stores it in a relational ledger, labels it, reports the result, and **audits itself** every night. It covers the whole life of a record: **capture** it, **process** it, **store** it, **notify** the outcome and **audit** what could not be resolved automatically.

It was built as three linked tracks: iOS Shortcuts that capture (Track 1), GitHub Actions scripts that ingest into Supabase (Track 2) and a nightly Claude routine that audits and teaches the system (Track 3). A setup interview turns the same code base into a template that any user configures with their own accounts, categories and fixed expenses.

Figure 1. Closed loop: from the tap on the phone to the nightly audit that improves the next record.

4. Problem It Solves

Expense tracking fails for two reasons: capturing a record costs too much effort at the moment of payment, and unattended automation tends to be wrong in ways nobody notices. The result is a spreadsheet that is behind reality, labels that drift, budgets that cannot be trusted and a credit card balance nobody can reconcile. The challenge is to make capture almost free *and* keep the data trustworthy without a human reviewing every row.

Risk Category	Concrete Manifestation
Capture friction	Opening an app, choosing fields and typing amounts at the till; people stop after a few weeks.
Silent failure	A broken integration drops a record and nobody notices until the monthly reconciliation.
Inconsistent labeling	The same product is labeled differently over time; category budgets stop meaning anything.
Unsafe automation	An AI model that writes categories, amounts or accounts directly can corrupt the budget.

Risk Category	Concrete Manifestation
Opaque credit debt	Card payments recorded as plain expenses make the debt per card impossible to read.
Operational risk	Tokens in scripts, destructive jobs, duplicated inserts after a retry.

5. Target Audience / Use Case

A single person (or a couple) managing several accounts and credit cards in one currency (EUR) who is comfortable creating a GitHub repository and a free Supabase project, plus freelancers and teams who want a reference implementation of a serverless data pipeline with an AI audit step. The system is configured per user with a Phase 0 interview and runs on free tiers; the only variable cost is the vision model, used only for receipt photos.

Data Model

Entity	Role in the model
Accounts	Spending, savings and credit-card accounts. A trigger derives the payment medium (debit or credit) from the account.
Categories (versioned)	Budget buckets with type, period and amount. Changing a budget closes the current row and opens a new one, so history keeps its own budget (SCD2).
Labels	Closed vocabulary with a criterion and keywords. A label lists the categories where it is allowed.
Ledger	One row per movement: amount, type, date, account, category, label, payment medium.
Pendings	Payloads that could not be recorded, with the error. Resolved overnight, then purged.
Templates	Fixed expenses, savings moves and card payments as versioned JSON, resolved by date.

Budget Policy

One budget amount is entered per period. The monthly view divides it by the months of the period. Yearly categories have their own view and one-off categories carry no periodic budget.

Period	Months	Monthly share	Where it shows
Monthly	1	budget	Monthly budget view
Every 2 / 3 / 4 / 6 months	2 / 3 / 4 / 6	budget / months	Monthly budget view
Yearly	12	n/a (yearly)	Annual budget view
One-off	none	n/a	No periodic budget

6. Solution Implemented

Three tracks, independently deployable but operationally linked. Track 1 removes friction, Track 2 guarantees that every record ends in the ledger or in a pending, and Track 3 turns the leftovers into learning. Guiding principle: **deterministic rules first, AI second, and AI never writes the budget**.

Track 1: Capture (iOS Shortcuts)

Shortcut	Trigger	What it does
Purchases	Wallet / NFC transaction	Pick account and categories, then upload a receipt photo or type amount, name and comment per category.
Manual hub	Manual	Income or expense; account, categories, with or without receipt photo; stamped with the system date.
Fixed expenses	Manual or time automation	Up to 7 one-tap fixed expenses, savings deposit / withdrawal and credit card payment.

Every branch ends in one HTTPS call to the GitHub repository_dispatch endpoint with a typed event (dynamic expense, photo expense or standardized ingestion) and a small JSON payload.

Track 2: Ingestion (GitHub Actions + Python)

Workflow	Trigger	Script	Purpose
gasto_dinamico	repository_dispatch	ingesta_dinamica.py	1 to 3 lines; label by category-bounded keyword matching.
gasto_foto	repository_dispatch	ingesta_foto.py	Receipt photo read by a vision model, glossary override, TOTAL check.
ingesta_estandarizada	repository_dispatch	ingesta_estandarizada.py	Fixed templates and linked rows (savings, card payment).
limpieza_recibos	cron, every 2 days	limpieza_recibos.py	Deletes receipt photos older than 7 days unless a pending still needs them.
purgar_pendientes	cron, daily	purgar_pendientes_procesados.py	Removes pendings already marked as processed.
tests	push / pull request	pytest + YAML check	Continuous integration of the whole code base.

Step-by-Step Decision Tree

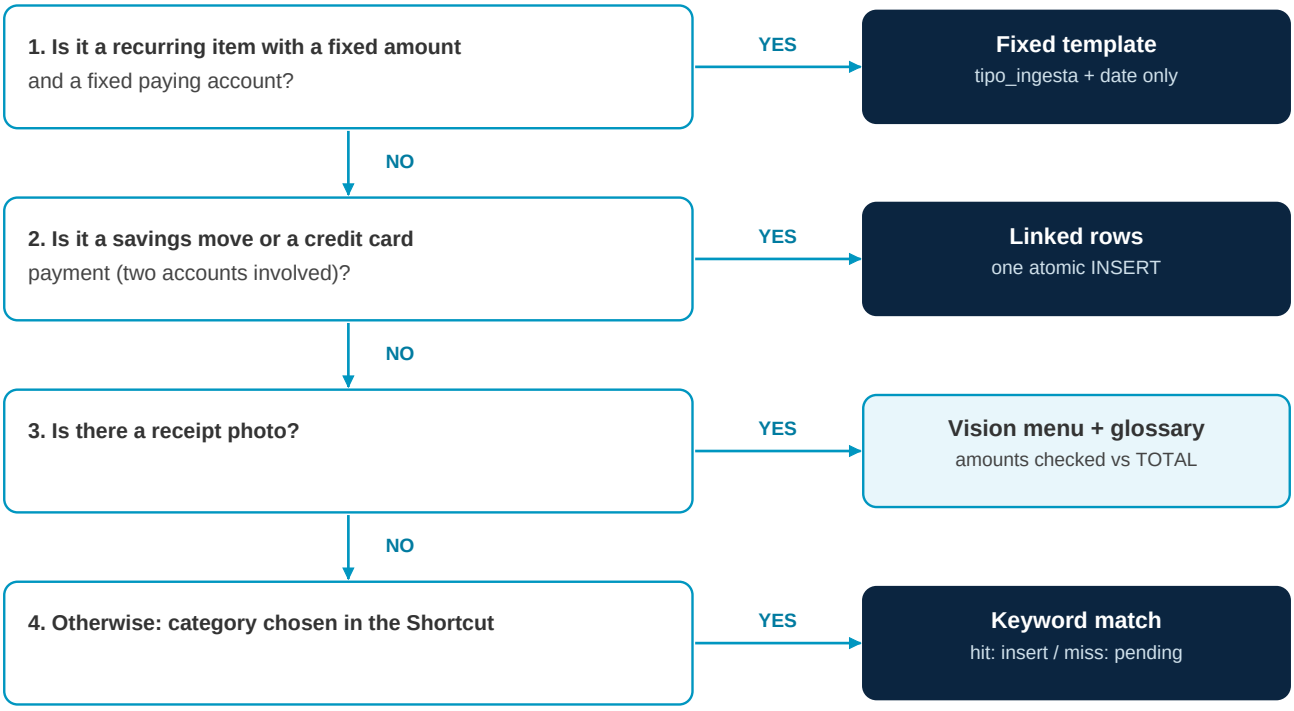

Figure 2. How a record is handled once it reaches the workflow.

Anti-Patterns the Design Prevents

Anti-pattern	Why it fails
Letting the AI pick amounts or categories freely	One hallucinated category corrupts a budget. Here the model only picks a number from a closed menu built from categories chosen by the user, and amounts are verified against the receipt TOTAL.
Fire-and-forget integrations	A failed call vanishes. Here every failure creates a pending record and a Telegram message with the reason.

Anti-pattern	Why it fails
Secrets in scripts or Shortcuts	A leaked token compromises the ledger. Here secrets live only in GitHub Secrets and Supabase Vault; the phone holds an insert-only key.
Destructive maintenance	Unattended deletes lose data. Here the routine only INSERTs and marks; a separate job purges what was marked.

Track 3: Audit (nightly Claude routine)

An unattended routine with the Supabase MCP connector closes the loop every night. It fixes a window of records (watermark to cap), then runs five phases. Its invariants: the category always rules, only the label may change on existing rows, nothing is deleted, each pending is inserted with a single statement, and duplicates are detected before inserting.

Phase	What happens	Output
0. Context	Reads the watermark and fixes the cap of the window.	Window of record ids
1. Pendings	Registers each pending (text, photo or template) with one INSERT and marks it processed.	Rows in the ledger
2. Validate	Judges each label against its criterion and siblings; corrects with an UPDATE of the label only.	Audit trail rows
3. Glossary	Extracts generic product keywords and adds them if no label has them.	New keywords
4. Log and notify	Writes the run log and sends the summary by Telegram and push.	Run log, message

Exception Lifecycle

Stage	What happens	Where
1. Capture	The record is sent with the typed event.	Track 1
2. Fail or no match	A pending is created with the payload and the reason; Telegram reports it.	Track 2
3. Resolve	The routine registers it, verifies the rows exist and marks it processed.	Track 3
4. Learn	New keywords reduce the chance of the same miss tomorrow.	Track 3
5. Purge	A daily job deletes processed pendings; old photos expire after 7 days.	Track 2

7. Technical Logic of Interest

7.1 Deterministic Labeling First, AI Second

The label is resolved by keyword matching against the live vocabulary, bounded to the labels allowed for the chosen category. The score is the length of the matching keywords, so a specific phrase beats a generic word. Only when nothing matches does the record become a pending for the nightly audit. It is instant, free and needs no network call.

7.2 Receipt Reading Verified Against the TOTAL

The vision model receives a numbered menu of label options limited to the categories picked on the phone and returns a number per product. The glossary overrides the model when keywords recognize the literal receipt text. The sum of lines must match the printed total within 0.05; otherwise exactly one re-read is requested with the mismatch as a hint, and a second failure becomes a pending instead of wrong numbers in the budget.

7.3 Versioned Categories (SCD2)

A category code can have several rows with validity dates; a partial unique index allows only one open row per code. Lookups use the validity date of the record, so a rent increase never rewrites last year's budget.

7.4 Linked Rows in One Statement

Savings moves and credit card payments are two rows with opposite signs inserted atomically: payer account minus, destination account (a savings account or a specific card) plus. Debt per card becomes a plain sum and any number of cards works.

7.5 Idempotent, Non-Destructive Audit

The routine works on a frozen window (watermark to cap), checks for an equivalent existing record before inserting and never deletes: it marks. A separate scheduled job, which uses the service key through REST and not the MCP connector, performs the purge.

7.6 Trust Boundaries

The service key exists only in GitHub Secrets. The phone holds a publishable key that the bucket policy restricts to INSERT. Every table has row-level security enabled with no policies, so only the service role reads or writes. Telegram credentials live in Vault and are used by a security-definer function that is not executable by the public roles.

7.7 Portable by Configuration

Nothing personal is hardcoded: time zone and currency symbol are repository variables, accounts, categories, labels and fixed expenses come from a setup interview that generates the seed and the templates, and the Shortcuts are published with placeholders.

8. Result / Impact

Results observed in the production instance used daily by the author and in the public template:

600+	36	8 + 4
ledger records in production use	automated tests, CI on every push	tables with row-level security + views

Metric	Value	Notes
Ledger records	600+	Real movements recorded through the Shortcuts
iOS Shortcuts	3	Purchases, manual hub, fixed expenses
GitHub workflows	5 + CI	3 ingestion, 2 maintenance, 1 test workflow
Database	8 tables, 4 views	Row-level security on all tables, 1 private bucket
Budget periods	7	Monthly to yearly plus one-off
Default label taxonomy	40	Criterion + keywords, extensible per user
Automated tests	36	Pure logic, cross-checks Shortcut / template / seed / workflow
Secrets in code	0	Verified by a scan before publication

Operational impact

Dimension	Before (spreadsheet)	After
Capture	Open file, type fields, often postponed.	One tap or an NFC payment; photo optional.
Failures	Lost silently.	Every failure becomes a pending with its reason.
Labeling	Manual and inconsistent.	Deterministic matching, audited nightly, glossary grows.
Budgets	Static, monthly only.	Versioned categories, seven periods, live views.
Credit cards	Debt estimated by hand.	Debt and available limit per card, always derived.

Dimension	Before (spreadsheet)	After
Security	Credentials spread across files.	Secrets in Secrets/Vault; insert-only key on the phone.

The system, the Shortcuts, the scripts and the documentation were delivered as a public template repository. No time-saving figures are published because they were not measured.

9. Technical Fragments

9.1 Libraries and Their Role

Library / Technology	Min. Version	Role in Project
Python	3.12	Language of all ingestion and maintenance scripts
requests	2.31+	HTTP calls to Supabase REST / Storage, Telegram and the vision API
PostgreSQL (Supabase)	17	Ledger, versioned categories, views, triggers, RLS
Supabase Storage / Vault / pg_net	managed	Private receipt bucket, secrets, Telegram call from SQL
GitHub Actions	n/a	Event-driven ingestion, cron maintenance, CI
Qwen3-VL (DashScope)	flash	Receipt reading through an OpenAI-compatible endpoint
Claude + MCP connector	n/a	Nightly audit routine with SQL tool access
pytest / PyYAML	8+ / 6+	Unit and cross-consistency tests, workflow YAML validation

9.2 Execution Flow

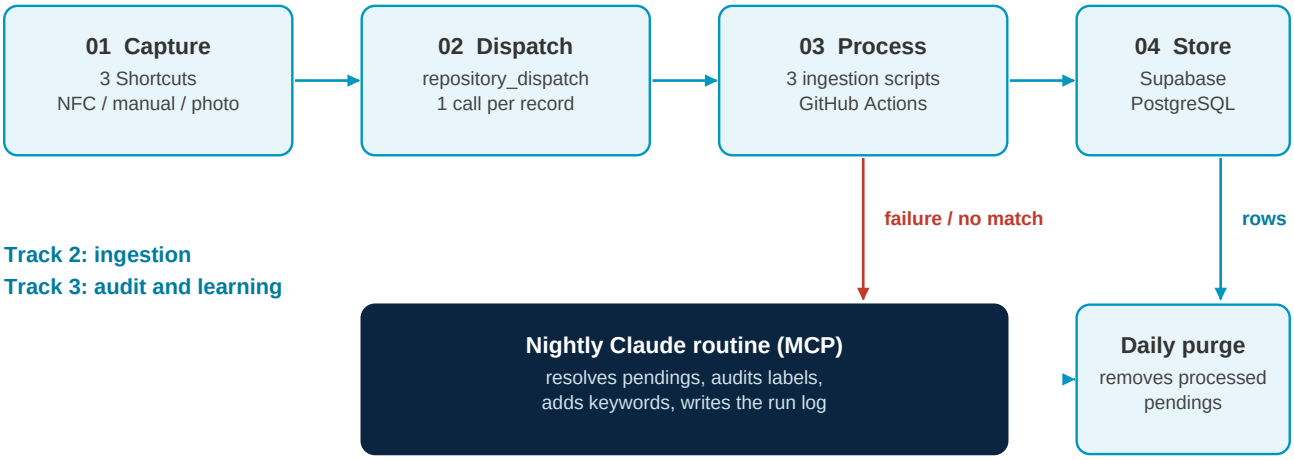

Figure 3. Event-driven ingestion with the audit loop behind it.

9.3 Execution Mode

Ingestion: serverless, event-driven; each record starts one short workflow run. **Maintenance:** two cron workflows (receipt cleanup every 2 days, pending purge daily). **Audit:** a scheduled Claude routine that runs unattended once per night, before the purge.

9.4 Key Code Snippets

Snippet 1: Keyword matching bounded by category (ingesta_dinamica.py, condensed)

```
def resolver_etiqueta_por_keyword(texto, codigo_categoria, etiquetas):
    texto_norm = normalizar(texto)
    mejor, mejor_score = None, 0
    for e in etiquetas:
        if codigo_categoria not in (e.get("posibles_categorias") or []):
            continue                      # the category rules
        score = sum(len(normalizar(kw)) for kw in (e.get("keyword") or [])
                    if normalizar(kw) and normalizar(kw) in texto_norm)
        if score > mejor_score:           # longer match wins
            mejor, mejor_score = e, score
    return (mejor["nombre_etiqueta"], mejor["id_etiqueta"]) if mejor else (None, None)
```

Snippet 2: Linked rows for savings and card payments (ingesta_estandarizada.py, condensed)

```
def construir_filas_multiples(version, fecha, monto, cuenta, cuenta_destino, comentario):
    monto_abs = abs(importe_a_float(monto))
    categoria = resolver_categoria_o_falla(version["codigo_categoria"], fecha)
    cuentas = {"cuenta": cuenta, "cuenta_destino": cuenta_destino}
    return [{
        "importe": monto_abs * fila["signo"],          # -amount on payer, +amount on destination
        "id_cuenta": resolver_cuenta(fila["id_cuenta"],
                                     cuentas[fila.get("cuenta_payload", "cuenta")]),
        "id_categoria": categoria["id_categoria"],
        "tipo": categoria["tipo"],
        "fecha_operacion": fecha,
    } for fila in version["filas"]]                    # inserted in ONE statement
```

Snippet 3: Credit position per card (supabase/setup.sql)

```
create or replace view public.vista_credito with (security_invoker = on) as
 select c.id_cuenta, c.nombre_cuenta, t.limite,
        coalesce(-sum(se.importe), 0)                  as deuda,
        t.limite - coalesce(-sum(se.importe), 0)       as credito_disponible
   from cuentas c
   join tarjeta_credito t on t.id_cuenta = c.id_cuenta
   left join seguimiento_efectivo se on se.id_cuenta = c.id_cuenta
  group by c.id_cuenta, c.nombre_cuenta, t.limite;
```

9.5 External API Dependencies

API	Key Endpoints	Auth Method	Use
GitHub REST	POST /repos/{owner}/{repo}/dispatches	Fine-grained token, this repo only	Trigger ingestion from the phone
Supabase REST (PostgREST)	/rest/v1/{table}	Service key (Actions only)	Resolve ids, insert rows, pendings
Supabase Storage	/storage/v1/object/{bucket}/{path}	Publishable key, insert-only policy	Receipt upload, read, cleanup
Telegram Bot API	/bot{token}/sendMessage	Bot token (Secrets / Vault)	Result and failure messages
DashScope (OpenAI-compatible)	/chat/completions	API key (Secrets)	Vision model for receipts

9.6 Error Handling and Logging

Every ingestion script keeps its core in a function that raises and a thin `main()` that decides: on any exception it stores a pending record with the payload and the error, sends the reason by Telegram and exits with a non-zero code so the workflow run is marked failed. Transient AI errors (5xx, 429) are retried a limited number of times; database errors are not retried, by design, because the pending and the nightly routine cover them. Workflow logs and the run log table provide the operational trace.

10. Privacy and Data Protection Review

PRIVACY REVIEW
The following elements were replaced by neutral placeholders in the public repository and in this document:
Supabase project reference and URLs: `{{SUPABASE_PROJECT_REF}}`
GitHub owner, repository and tokens: `{{GITHUB_OWNER}}, {{GITHUB_REPO}}, {{GITHUB_PAT}}`
Account, card and category names, Wallet card identifiers: `{{ACCOUNT_n}}, {{CREDIT_CARD_n}}, {{CATEGORY_n}}`
Telegram bot token and chat identifier: stored only in GitHub Secrets and Supabase Vault.

No real transactions, balances, credentials or personal identifiers appear in this document or in the repository; figures are aggregate counts only.